

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ABHISHEK(1BM25CS404)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **ABHISHEK(1BM25CS404)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of **Analysis and Design of Algorithms Lab - (23CS4PCADA)** work prescribed for the said degree.

Dr. Shwetha K S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	9
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	13
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	16
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	19
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	24
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	27
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	32
9	Implement Fractional Knapsack using Greedy technique.	38
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	43
11	Implement "N-Queens Problem" using Backtracking.	46

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1: Topological Sorting

```
#include <stdio.h>
```

```
#define MAX 6
```

```
int main() {
```

```
    int adj[MAX][MAX] = {
```

```
        {0, 1, 1, 0, 0, 0},
```

```
        {0, 0, 0, 1, 0, 0},
```

```
        {0, 0, 0, 1, 1, 0},
```

```
        {0, 0, 0, 0, 0, 1},
```

```
        {0, 0, 0, 0, 0, 1},
```

```
        {0, 0, 0, 0, 0, 0}
```

```
    };
```

```
    int indegree[MAX] = {0};
```

```
    int removed[MAX] = {0};
```

```
    for (int i = 0; i < MAX; i++) {
```

```
        for (int j = 0; j < MAX; j++) {
```

```
            if (adj[i][j] == 1) {
```

```
                indegree[j]++;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("Topological Order: ");
```

```
    for (int count = 0; count < MAX; count++) {
```

```
        int found = -1;
```

```

for (int i = 0; i < MAX; i++) {
    if (indegree[i] == 0 && removed[i] == 0) {
        found = i;
        break;
    }
}

if (found == -1) {
    printf("\nGraph has a cycle!\n");
    return 0;
}

printf("%d ", found);
removed[found] = 1;

for (int j = 0; j < MAX; j++) {
    if (adj[found][j] == 1) {
        indegree[j]--;
    }
}

return 0;
}

```

Output :

Topological Order: 0 1 2 3 4 5

LeetCode 207: Course Schedule

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
```

```
    int** adj = (int**)malloc(numCourses * sizeof(int*));
```

```
    int* adjSize = (int*)calloc(numCourses, sizeof(int));
```

```
    for (int i = 0; i < numCourses; i++) {
```

```
        adj[i] = (int*)malloc(prerequisitesSize * sizeof(int));
```

```
    }
```

```
    int* indegree = (int*)calloc(numCourses, sizeof(int));
```

```
    for (int i = 0; i < prerequisitesSize; i++) {
```

```
        int a = prerequisites[i][0];
```

```
        int b = prerequisites[i][1];
```

```
        adj[b][adjSize[b]++] = a;
```

```
        indegree[a]++;
```

```
    }
```

```
    int* queue = (int*)malloc(numCourses * sizeof(int));
```

```
    int front = 0, rear = 0;
```

```
    for (int i = 0; i < numCourses; i++) {
```

```
        if (indegree[i] == 0) {
```

```
            queue[rear++] = i;
```

```
        }
```

```
    }
```

```
    int count = 0;
```

```
    while (front < rear) {
```

```

    int curr = queue[front++];
    count++;

    for (int i = 0; i < adjSize[curr]; i++) {
        int neighbor = adj[curr][i];
        indegree[neighbor]--;
        if (indegree[neighbor] == 0) {
            queue[rear++] = neighbor;
        }
    }
}

for (int i = 0; i < numCourses; i++) {free(adj[i]);}
free(adj);
free(adjSize);
free(indegree);
free(queue);

return count == numCourses;
}

```

Output :

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

numCourses =

2

prerequisites =

[[1,0]]

Output

true

Expected

true

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

numCourses =

2

prerequisites =

[[1,0],[0,1]]

Output

false

Expected

false

Lab program 2: Johnson Trotter

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

struct Element {
    int value;
    char dir;
};

struct Element perm[MAX];

void printPermutation(int n) {
    for (int i = 0; i < n; i++) {printf("%d ", perm[i].value);}
    printf("\n");
}

int getLargestMobile(int n) {
    int largestMobile = -1;
    int index = -1;

    for (int i = 0; i < n; i++) {
        int nextIndex;
        if (perm[i].dir == '<')
            nextIndex = i - 1;
        else
            nextIndex = i + 1;

        if (nextIndex >= 0 && nextIndex < n) {
            if (perm[i].value > perm[nextIndex].value) {
```

```

        if (perm[i].value > largestMobile) {
            largestMobile = perm[i].value;
            index = i;
        }
    }
}
return index;
}

```

```

void reverseDirection(int n, int value) {
    for (int i = 0; i < n; i++) {
        if (perm[i].value > value) {
            if (perm[i].dir == '<')
                perm[i].dir = '>';
            else
                perm[i].dir = '<';
        }
    }
}

```

```

void johnsonTrotter(int n) {
    for (int i = 0; i < n; i++) {
        perm[i].value = i + 1;
        perm[i].dir = '<';
    }
}

```

```

printPermutation(n);

```

```

while (1) {

```

```

int mobileIndex = getLargestMobile(n);

if (mobileIndex == -1)
    break;

int swapIndex;
if (perm[mobileIndex].dir == '<')
    swapIndex = mobileIndex - 1;
else
    swapIndex = mobileIndex + 1;

struct Element temp = perm[mobileIndex];
perm[mobileIndex] = perm[swapIndex];
perm[swapIndex] = temp;

mobileIndex = swapIndex;

reverseDirection(n, perm[mobileIndex].value);
printPermutation(n);
}
}

int main() {
    int n;
    printf("Enter number of elements: ");scanf("%d", &n);

    if (n > MAX) {
        printf("Max allowed is %d\n", MAX);
        return 1;
    }
}

```

```
johnsonTrotter(n);  
return 0;  
}
```

Output :

Enter number of elements: 3

1 2 3

1 3 2

3 1 2

3 2 1

2 3 1

2 1 3

Lab program 3: Merge Sort

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void merge(int arr[], int l, int m, int r) {
```

```
    int i, j, k;
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    int *L = (int *)malloc(n1 * sizeof(int));
```

```
    int *R = (int *)malloc(n2 * sizeof(int));
```

```
    for (i = 0; i < n1; i++) {L[i] = arr[l + i];}
```

```
    for (j = 0; j < n2; j++) {R[j] = arr[m + 1 + j];}
```

```
    i = 0; j = 0; k = l;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {arr[k++] = L[i++];}
```

```
        else {arr[k++] = R[j++];}
```

```
    }
```

```
    while (i < n1) {arr[k++] = L[i++];}
```

```
    while (j < n2) {arr[k++] = R[j++];}
```

```
    free(L);
```

```
    free(R);
```

```
}
```

```
void mergeSort(int arr[], int l, int r) {
```

```

    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void runTest(int size) {
    int *arr = (int *)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {arr[i] = rand();}

    clock_t start = clock();
    mergeSort(arr, 0, size - 1);
    clock_t end = clock();

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Array Size: %d\t\t Time Taken: %f seconds\n", size, time_taken);
    free(arr);
}

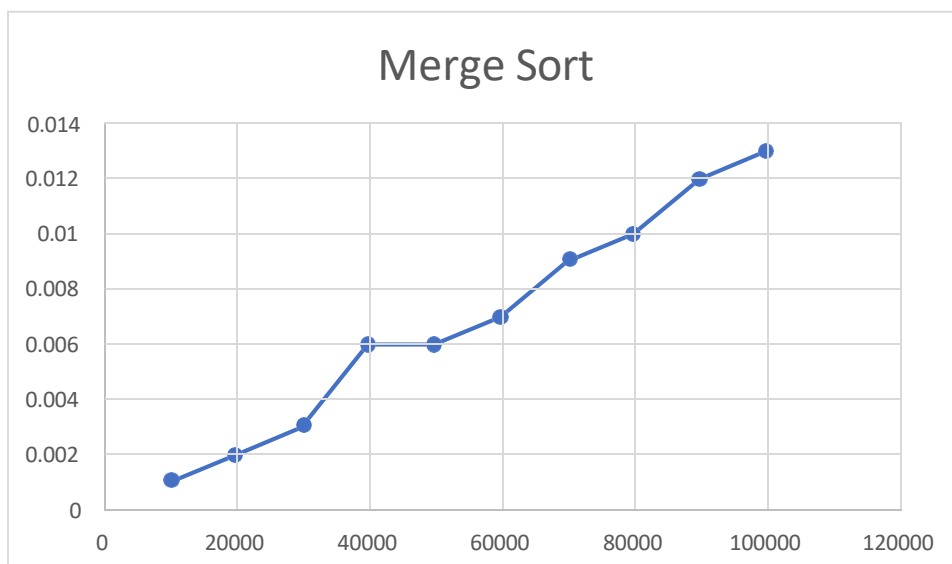
int main() {
    srand(time(NULL));
    double sizes[] = {1e4, 2e4, 3e4, 4e4, 5e4, 6e4, 7e4, 8e4, 9e4, 1e5};
    int n = sizeof(sizes) / sizeof(sizes[0]);

    for (int i = 0; i < n; i++) {runTest(sizes[i]);}
    return 0;
}

```

Output :

Array Size: 10000	Time Taken: 0.000000 seconds
Array Size: 20000	Time Taken: 0.012000 seconds
Array Size: 30000	Time Taken: 0.020000 seconds
Array Size: 40000	Time Taken: 0.023000 seconds
Array Size: 50000	Time Taken: 0.034000 seconds
Array Size: 60000	Time Taken: 0.040000 seconds
Array Size: 70000	Time Taken: 0.044000 seconds
Array Size: 80000	Time Taken: 0.065000 seconds
Array Size: 90000	Time Taken: 0.048000 seconds
Array Size: 100000	Time Taken: 0.076000 seconds



Lab program 4: Quick Sort

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void swap(int* a, int* b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for(int j = low; j < high; j++) {
```

```
        if(arr[j] <= pivot) {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    }
```

```
    swap(&arr[i + 1], &arr[high]);
```

```
    return i + 1;
```

```
}
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if(low < high) {
```

```
        int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
```



```

    }
}

void runTest(int size) {
    int* arr = (int*)malloc(size * sizeof(int));
    for(int i = 0; i < size; i++) {arr[i] = rand();}

    clock_t start = clock();
    quickSort(arr, 0, size - 1);
    clock_t end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Array Size: %d\t\t Time Taken: %f seconds\n", size, time_taken);
    free(arr);
}

int main() {
    srand(time(NULL));

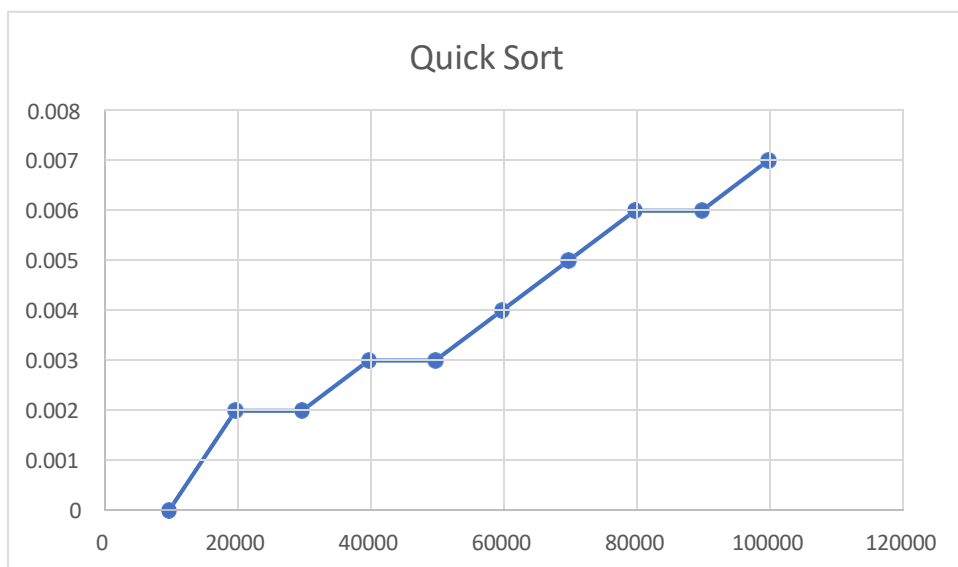
    double sizes[] = {1e4, 2e4, 3e4, 4e4, 5e4, 6e4, 7e4, 8e4, 9e4, 1e5};
    int n = sizeof(sizes) / sizeof(sizes[0]);

    for(int i = 0; i < n; i++) {runTest(sizes[i]);}
    return 0;
}

```

Output :

Array Size: 10000	Time Taken: 0.002000 seconds
Array Size: 20000	Time Taken: 0.003000 seconds
Array Size: 30000	Time Taken: 0.000000 seconds
Array Size: 40000	Time Taken: 0.022000 seconds
Array Size: 50000	Time Taken: 0.011000 seconds
Array Size: 60000	Time Taken: 0.022000 seconds
Array Size: 70000	Time Taken: 0.016000 seconds
Array Size: 80000	Time Taken: 0.034000 seconds
Array Size: 90000	Time Taken: 0.028000 seconds
Array Size: 100000	Time Taken: 0.029000 seconds



Lab program 5: Heap Sort

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void heapify(int arr[], int n, int i) {  
    int largest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
  
    if (left < n && arr[left] > arr[largest]) {largest = left;}  
    if (right < n && arr[right] > arr[largest]) {largest = right;}  
    if (largest != i) {  
        int temp = arr[i];  
        arr[i] = arr[largest];  
        arr[largest] = temp;  
        heapify(arr, n, largest);  
    }  
}
```

```
void heapSort(int arr[], int n) {  
    for (int i = n / 2 - 1; i >= 0; i--) {heapify(arr, n, i);}  
    for (int i = n - 1; i > 0; i--) {  
        int temp = arr[0];  
        arr[0] = arr[i];  
        arr[i] = temp;  
        heapify(arr, i, 0);  
    }  
}
```

```

void runTest(int size) {
    int *arr = (int *)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {arr[i] = rand();}

    clock_t start = clock();
    heapSort(arr, size);
    clock_t end = clock();

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Array Size: %d\t\t Time Taken: %f seconds\n",size, time_taken);
    free(arr);
}

int main() {
    srand(time(NULL));

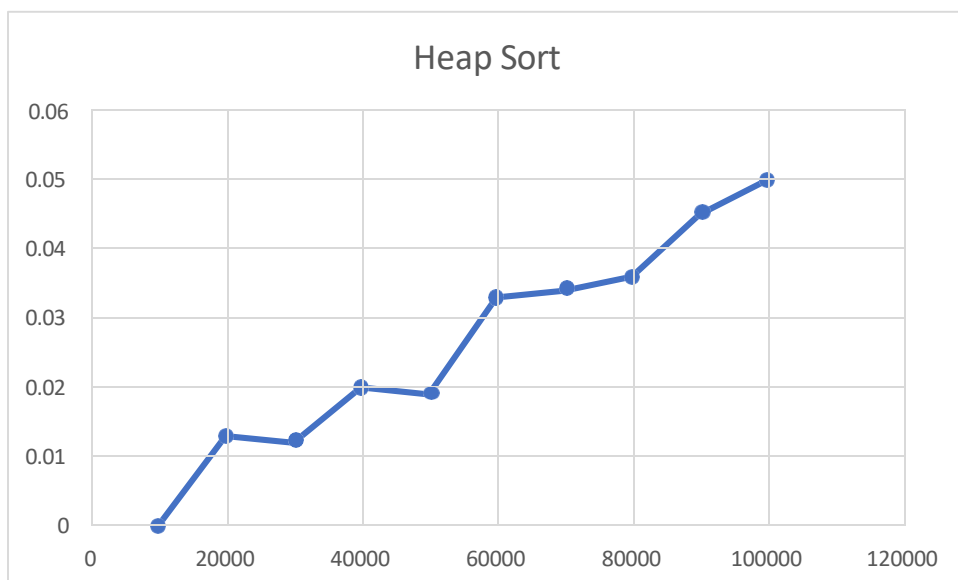
    double sizes[] = {1e4, 2e4, 3e4, 4e4, 5e4, 6e4, 7e4, 8e4, 9e4, 1e5};
    int n = sizeof(sizes) / sizeof(sizes[0]);

    for(int i = 0; i < n; i++) {runTest(sizes[i]);}
    return 0;
}

```

Output :

Array Size: 10000	Time Taken: 0.000000 seconds
Array Size: 20000	Time Taken: 0.013000 seconds
Array Size: 30000	Time Taken: 0.012000 seconds
Array Size: 40000	Time Taken: 0.020000 seconds
Array Size: 50000	Time Taken: 0.019000 seconds
Array Size: 60000	Time Taken: 0.033000 seconds
Array Size: 70000	Time Taken: 0.034000 seconds
Array Size: 80000	Time Taken: 0.036000 seconds
Array Size: 90000	Time Taken: 0.045000 seconds
Array Size: 100000	Time Taken: 0.050000 seconds



LeetCode 912: Sort an Array

```
void swap(int* a, int* b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
void heapify(int* nums, int n, int i) {  
    int largest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
    if (left < n && nums[left] > nums[largest])  
        largest = left;  
  
    if (right < n && nums[right] > nums[largest])  
        largest = right;  
  
    if (largest != i) {  
        swap(&nums[i], &nums[largest]);  
        heapify(nums, n, largest);  
    }  
}
```

```
int* sortArray(int* nums, int numsSize, int* returnSize) {  
    *returnSize = numsSize;  
  
    for (int i = numsSize / 2 - 1; i >= 0; i--) {  
        heapify(nums, numsSize, i);  
    }  
  
    for (int i = numsSize - 1; i > 0; i--) {
```

```
        swap(&nums[0], &nums[i]);
        heapify(nums, i, 0);
    }

    return nums;
}
```

Output:

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

```
nums =
[5,2,3,1]
```

Output

```
[1,2,3,5]
```

Expected

```
[1,2,3,5]
```

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

```
nums =
[5,1,1,2,0,0]
```

Output

```
[0,0,1,1,2,5]
```

Expected

```
[0,0,1,1,2,5]
```

Lab program 6: 0/1 Knapsack

```
#include <stdio.h>

int main() {
    int weights[] = {1, 3, 4, 5};
    int values[] = {1, 4, 5, 7};
    int n = 4;
    int capacity = 7;
    int dp[n + 1][capacity + 1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            if (i == 0 || w == 0) {dp[i][w] = 0;}
            else if (weights[i - 1] <= w) {
                int include = values[i - 1] + dp[i - 1][w - weights[i - 1]];
                int exclude = dp[i - 1][w];

                dp[i][w] = (include > exclude) ? include : exclude;
            }
            else {dp[i][w] = dp[i - 1][w];}
        }
    }

    printf("Maximum value in Knapsack = %d\n", dp[n][capacity]);
    return 0;
}
```

Output :

Maximum value in Knapsack = 9

LeetCode 494: Target Sum

```
int findTargetSumWays(int* nums, int numsSize, int target) {  
    int sum = 0;  
    for (int i = 0; i < numsSize; i++) {sum += nums[i];}  
    if (target > sum || target < -sum) return 0;  
  
    int offset = sum;  
    int dp[numsSize + 1][2 * sum + 1];  
  
    memset(dp, 0, sizeof(dp));  
    dp[0][offset] = 1;  
  
    for (int i = 1; i <= numsSize; i++) {  
        for (int j = 0; j <= 2 * sum; j++) {  
            if (dp[i-1][j] != 0) {  
                if (j + nums[i-1] <= 2 * sum)  
                    dp[i][j + nums[i-1]] += dp[i-1][j];  
                if (j - nums[i-1] >= 0)  
                    dp[i][j - nums[i-1]] += dp[i-1][j];  
            }  
        }  
    }  
    return dp[numsSize][target + offset];  
}
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

```
nums =  
[1,1,1,1,1]
```

```
target =  
3
```

Output

5

Expected

5

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

```
nums =  
[1]
```

```
target =  
1
```

Output

1

Expected

1

Lab program 7: Floyd's algorithm

```
#include <stdio.h>

#define INF 99999

#define V 4

void floydWarshall(int graph[V][V]) {
    int dist[V][V], i, j, k;
    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            dist[i][j] = graph[i][j];
        }
    }

    for (k = 0; k < V; k++) {
        for (i = 0; i < V; i++) {
            for (j = 0; j < V; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    printf("The shortest distances between every pair of vertices are:\n");
    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            if (dist[i][j] == INF) {
                printf("%4s", "INF");
            } else {
                printf("%4d", dist[i][j]);
            }
        }
    }
}
```

```

    }
    printf("\n");
}
}

int main() {
    int graph[V][V] = {
        {0, 3, INF, INF},
        {2, 0, INF, INF},
        {INF, 7, 0, 1},
        {6, INF, INF, 0}
    };
    floydWarshall(graph);
    return 0;
}

```

Output :

The shortest distances between every pair of vertices are:

```

0  3 INF INF
2  0 INF INF
7  7  0  1
6  9 INF  0

```

LeetCode 1334: Find the City With the Smallest Number of Neighbors at a Threshold Distance

```
#define INF 999999
```

```
int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold) {  
    int dist[n][n];
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            if (i == j)  
                dist[i][j] = 0;  
            else  
                dist[i][j] = INF;  
        }  
    }
```

```
    for (int i = 0; i < edgesSize; i++) {  
        int u = edges[i][0];  
        int v = edges[i][1];  
        int w = edges[i][2];  
  
        dist[u][v] = w;  
        dist[v][u] = w;  
    }
```

```
    for (int k = 0; k < n; k++) {  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                if (dist[i][k] + dist[k][j] < dist[i][j]) {  
                    dist[i][j] = dist[i][k] + dist[k][j];  
                }  
            }  
        }  
    }
```

```

        }
    }
}

int minReachable = INT_MAX;
int resultCity = -1;

for (int i = 0; i < n; i++) {
    int count = 0;

    for (int j = 0; j < n; j++) {
        if (i != j && dist[i][j] <= distanceThreshold) {
            count++;
        }
    }

    if (count <= minReachable) {
        minReachable = count;
        resultCity = i;
    }
}

return resultCity;
}

```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

n =

4

edges =

[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

distanceThreshold =

4

Output

3

Expected

3

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

n =

5

edges =

[[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]

distanceThreshold =

2

Output

0

Expected

0

Lab program 8a: Prim's algorithm

```
#include <stdio.h>

#define V 5

int minKey(int key[], int mstSet[]) {
    int min = 9999, min_index;
    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void printMST(int parent[], int graph[V][V]) {
    int totalCost = 0;
    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }
    printf("Total Cost = %d\n", totalCost);
}

void primMST(int graph[V][V]) {
    int parent[V]; int key[V]; int mstSet[V];
    for (int i = 0; i < V; i++) {key[i] = 9999; mstSet[i] = 0;}
    key[0] = 0; parent[0] = -1;
```



```

for (int count = 0; count < V - 1; count++) {
    int u = minKey(key, mstSet);
    mstSet[u] = 1;
    for (int v = 0; v < V; v++) {
        if (graph[u][v] &&
            mstSet[v] == 0 &&
            graph[u][v] < key[v]) {
            parent[v] = u;
            key[v] = graph[u][v];
        }
    }
}
printMST(parent, graph);
}

```

```

int main() {
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };
    primMST(graph);
    return 0;
}

```

Output :

Edge	Weight
------	--------

0 - 1	2
-------	---

1 - 2	3
-------	---

0 - 3	6
-------	---

1 - 4	5
-------	---

Total Cost = 16

Lab program 8b: Kruskal's algorithm

```
#include <stdio.h>

int parent[10];

int find(int i){
    while(parent[i] != i)
        i = parent[i];
    return i;
}

void uni(int a, int b){parent[a] = b;}

int main(){
    int n = 4;
    int cost[4][4] = {
        {999, 10, 6, 5},
        {10, 999, 999, 15},
        {6, 999, 999, 4},
        {5, 15, 4, 999}
    };

    int i, j, min, a, b;
    int edges = 0;
    int total = 0;

    for(i = 0; i < n; i++)
        parent[i] = i;

    printf("Edges in MST:\n");

    while(edges < n - 1){
```

```

min = 999;
for(i = 0; i < n; i++){
    for(j = 0; j < n; j++){
        if(cost[i][j] < min){
            min = cost[i][j];
            a = i;
            b = j;
        }
    }
}

int u = find(a);
int v = find(b);

if(u != v){
    printf("%d - %d = %d\n", a, b, min);
    total += min;
    uni(u, v);
    edges++;
}
cost[a][b] = cost[b][a] = 999;
}

printf("Total cost = %d\n", total);
return 0;
}

```

Output :

Edges in MST:

$$2 - 3 = 4$$

$$0 - 3 = 5$$

$$0 - 1 = 10$$

$$\text{Total cost} = 19$$

Lab program 9: Fractional Knapsack

```
#include <stdio.h>
```

```
struct Item {  
    int weight;  
    int profit;  
    float ratio;  
};
```

```
void sortItems(struct Item items[], int n) {  
    int i, j;  
    struct Item temp;  
  
    for (i = 0; i < n - 1; i++) {  
        for (j = i + 1; j < n; j++) {  
            if (items[i].ratio < items[j].ratio) {  
                temp = items[i];  
                items[i] = items[j];  
                items[j] = temp;  
            }  
        }  
    }  
}
```

```
int main() {  
    int capacity = 50;  
    int n = 3;  
    struct Item items[3] = {  
        {10, 60, 0},  
        {20, 100, 0},  
        {30, 120, 0}  
    }  
}
```

```

};

int i;
float totalProfit = 0.0;

for (i = 0; i < n; i++) {items[i].ratio = (float)items[i].profit / items[i].weight;}

sortItems(items, n);
printf("Selected items:\n");
for (i = 0; i < n; i++) {
    if(capacity >= items[i].weight) {
        capacity -= items[i].weight;
        totalProfit += items[i].profit;
        printf("Take full item -> Weight: %d Profit: %d\n",
            items[i].weight, items[i].profit);
    }
    else {
        float fraction = (float)capacity / items[i].weight;
        totalProfit += items[i].profit * fraction;
        printf("Take %.2f fraction of item -> Weight: %d Profit: %d\n",
            fraction, items[i].weight, items[i].profit);

        capacity = 0;
        break;
    }
}

printf("\nMaximum Profit = %.2f\n", totalProfit);
return 0;
}

```

Output :

Selected items:

Take full item -> Weight: 10 Profit: 60

Take full item -> Weight: 20 Profit: 100

Take 0.67 fraction of item -> Weight: 30 Profit: 120

Maximum Profit = 240.00

LeetCode 316: Remove Duplicate Letters

```
char* removeDuplicateLetters(char* s) {  
    int count[256] = {0};  
    bool visited[256] = {0};  
    int len = strlen(s);  
  
    for (int i = 0; i < len; i++) {count[(unsigned char)s[i]]++;}  
  
    char* stack = (char*)malloc(len + 1);  
    int top = -1;  
  
    for (int i = 0; i < len; i++) {  
        char c = s[i];  
        count[(unsigned char)c]--;  
  
        if (visited[(unsigned char)c])  
            continue;  
  
        while (top >= 0 && stack[top] > c && count[(unsigned char)stack[top]] > 0) {  
            visited[(unsigned char)stack[top]] = false;  
            top--;  
        }  
  
        stack[++top] = c;  
        visited[(unsigned char)c] = true;  
    }  
  
    stack[top + 1] = '\0';  
    return stack;  
}
```

Output:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

s =
"bcabc"

Output

"abc"

Expected

"abc"

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

s =
"cbacdcbc"

Output

"acdb"

Expected

"acdb"

Lab program 10: Dijkstra's algorithm.

```
#include <stdio.h>

#define V 6
#define INF 99999

int minDistance(int dist[], int visited[]) {
    int min = INF, min_index = -1;

    for (int v = 0; v < V; v++) {
        if (visited[v] == 0 && dist[v] < min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

void dijkstra(int graph[V][V], int src) {
    int dist[V];
    int visited[V];

    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited);
        visited[u] = 1;
```

```

    for (int v = 0; v < V; v++) {
        if (!visited[v] &&
            graph[u][v] &&
            dist[u] != INF &&
            dist[u] + graph[u][v] < dist[v]) {

            dist[v] = dist[u] + graph[u][v];
        }
    }
}

printf("Vertex\tDistance from Source\n");
for (int i = 0; i < V; i++) {printf("%d \t\t %d\n", i, dist[i]);}
}

int main() {
    int graph[V][V] = {
        {0, 4, 0, 0, 0, 0},
        {4, 0, 8, 0, 0, 0},
        {0, 8, 0, 7, 2, 4},
        {0, 0, 7, 0, 9, 14},
        {0, 0, 2, 9, 0, 10},
        {0, 0, 4, 14, 10, 0}
    };

    int source = 0;
    dijkstra(graph, source);
    return 0;
}

```

Output :

Vertex Distance from Source

0	0
1	4
2	12
3	19
4	14
5	16

Lab program 11: N-Queens Problem

```
#include <stdio.h>

#define N 4

int board[N][N];

void printBoard() {
    int i, j;

    for(i = 0; i < N; i++) {
        for(j = 0; j < N; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

int isSafe(int row, int col) {
    int i, j;

    for(i = 0; i < col; i++) {
        if(board[row][i])
            return 0;
    }

    for(i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if(board[i][j])
            return 0;
    }

    for(i = row, j = col; i < N && j >= 0; i++, j--) {
        if(board[i][j])
```

```

        return 0;
    }

    return 1;
}

int solveNQ(int col) {
    int i;

    if(col >= N)
        return 1;

    for(i = 0; i < N; i++) {
        if(isSafe(i, col)) {
            board[i][col] = 1;
            if(solveNQ(col + 1))
                return 1;
            board[i][col] = 0;
        }
    }

    return 0;
}

int main() {
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            board[i][j] = 0;
        }
    }
}

```

```
if(solveNQ(0)) {  
    printf("Solution for %d-Queens Problem:\n\n", N);  
    printBoard();  
}  
else {printf("No solution exists");}  
return 0;  
}
```

Output :

Solution for 4-Queens Problem:

```
0 0 1 0  
1 0 0 0  
0 0 0 1  
0 1 0 0
```